Лабораторна робота 2

1 Мета та постановка задачі

Дослідити вплив порога класифікації на вартість помилок незмінної моделі виявлення шахрайських транзакцій. Пропуск шахрайства коштує 10 умовних одиниць, хибна тривога - 1. Поріг вибирається лише на validation; test використовується після фіксації порога та проходження незалежних перевірок.

2 Дані, поділ і стандартизація

У вихідному наборі 284 807 транзакцій, 492 шахрайські. Class = 1 - шахрайство, Class = 0 - легітимна транзакція. Використано 29 ознак: V1–V28 і Amount. Time та Class не входять до ознак. Сирий Amount збережено окремо і не включено до вартості помилок.

Порядок CSV не змінюється перед поділом. Ідентифікатор — номер рядка від 0 без заголовка. Спочатку train_test_split ділить індекси з test_size=0.4, stratify=y, random_state=0. Потім відкладені індекси діляться навпіл із test_size=0.5, стратифікацією та random_state=0; перша частина - validation, друга - test. Балансування класів не застосовується.

Для частини вибірки S її розмір, кількість шахрайств і частка шахрайств:

$$n_S = |S|, P_S = \sum_{i \in S} y_i \,, \pi_S = \frac{P_S}{n_S}.$$

Після поділу отримано такі кількості об'єктів і частки шахрайств. У таблицях округлено лише відображення; CSV та JSON зберігають повну точність.

split	n	positives	positive_fraction
train	170884	295	0.001726317268
validation	56961	98	0.001720475413
test	56962	99	0.001738000772

Для ознаки j середнє і дисперсія обчислюються лише за train (N — кількість навчальних об'єктів, ddof = 0):

$$\mu_j = \frac{1}{N}\sum_{i \in train} x_{ij} \,, \sigma_j^2 = \frac{1}{N}\sum_{i \in train} (x_{ij} - \mu_j)^2, \sigma_j = \sqrt{\sigma_j^2}.$$

Стандартизована ознака:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{a_j}, a_j = \begin{cases} \sigma_j, & \sigma_j > 0, \\ 1, & \sigma_j = 0. \end{cases}$$

Використовується StandardScaler (він також захищає масштабування практично сталих ознак). Validation і test використовують незмінні статистики train.

Стандартизація на всьому наборі передала б інформацію про розподіл відкладених даних у підготовку моделі — витік даних. Матриці моделі перетворено у float32.

3 Модель, прямий прохід і навчання

Архітектура: Linear(29,32) → ReLU → Linear(32,1). Для пакета з m об'єктів X має форму m × 29. У PyTorch W_1 має форму 32 × 29, b_1 — 32; W_2 — 1 × 32, b_2 — 1. Зсуви додаються до кожного рядка:

$$H = XW_1^T + \mathbf{1}_m b_1^T,$$

$$A = ReLU(H),$$

$$z = AW_2^T + \mathbf{1}_m b_2,$$

Логіт z перетворюється в оцінку s:

$$s_i = sigmoid(z_i) = \frac{1}{1 + e^{-z_i}}.$$

Кількість параметрів: 29·32 + 32 + 32·1 + 1 = 993. Стандартна ініціалізація nn.Linear: для шару з d входами ваги та зсуви беруться з рівномірного розподілу U(−1/√d, 1/√d). Перед створенням моделі викликається torch.manual_seed(0).

Незважена середня бінарна крос-ентропія:

$$L = -\frac{1}{m}\sum_{i=1}^{m}[y_i \ln s_i + (1 - y_i)\ln(1 - s_i)].$$

У коді використано BCEWithLogitsLoss, що обчислює математично еквівалентну стабільну форму з логітів:

$$L = \frac{1}{m}\sum_{i=1}^{m}\left[max(z_i, 0) - y_i z_i + \ln(1 + e^{-|z_i|})\right].$$

Функція втрат отримує логіти, без попереднього sigmoid. Число m — фактичний розмір поточного пакета, включно з останнім неповним пакетом. Середня втрата епохи для журналу: L_epoch = (Σ_b m_b L_b) / N, де b нумерує пакети.

4 Метрики і ціна рішення

Правило класифікації для порога t:

$$\hat{y}_i(t) = \mathbf{1}[s_i \geq t].$$

Лічильники реалізовано в thresholds.py:

$$TP(t) = \sum_i \mathbf{1}[y_i = 1 \wedge s_i \geq t], FP(t) = \sum_i \mathbf{1}[y_i = 0 \wedge s_i \geq t],$$

$$FN(t) = \sum_i \mathbf{1}[\, y_i = 1 \wedge s_i < t], TN(t) = \sum_i \mathbf{1}[\, y_i = 0 \wedge s_i < t].$$

$$C(t) = 10FN(t) + FP(t),$$

$$precision(t) = \frac{TP(t)}{TP(t) + FP(t)}$$

$$recall(t) = \frac{TP(t)}{TP(t) + FN(t)}.$$

Precision — частка справжніх шахрайств серед спрацювань; recall — частка знайдених шахрайств серед усіх шахрайств. Якщо відповідний знаменник нульовий, метрика не визначена: немає прогнозованих позитивних об'єктів або немає істинних позитивних об'єктів.

5 PR, ROC та ROC-AUC

PR-крива показує precision як функцію recall при зміні порога. ROC використовує:

$$TPR(t) = \frac{TP(t)}{TP(t) + FN(t)}, FPR(t) = \frac{FP(t)}{FP(t) + TN(t)}.$$

Криві обчислюються функціями scikit-learn. Кінцева точка PR з precision = 1 і recall = 0 - умовність побудови кривої, а не визначена precision для рішення без спрацювань. ROC-AUC - площа під ROC, для точок зі зростанням FPR:

$$AUC = \sum_{j=1}^{K-1} (FP\, R_{j+1} - FPR_j) \frac{TPR_{j+1} + TPR_j}{2}.$$

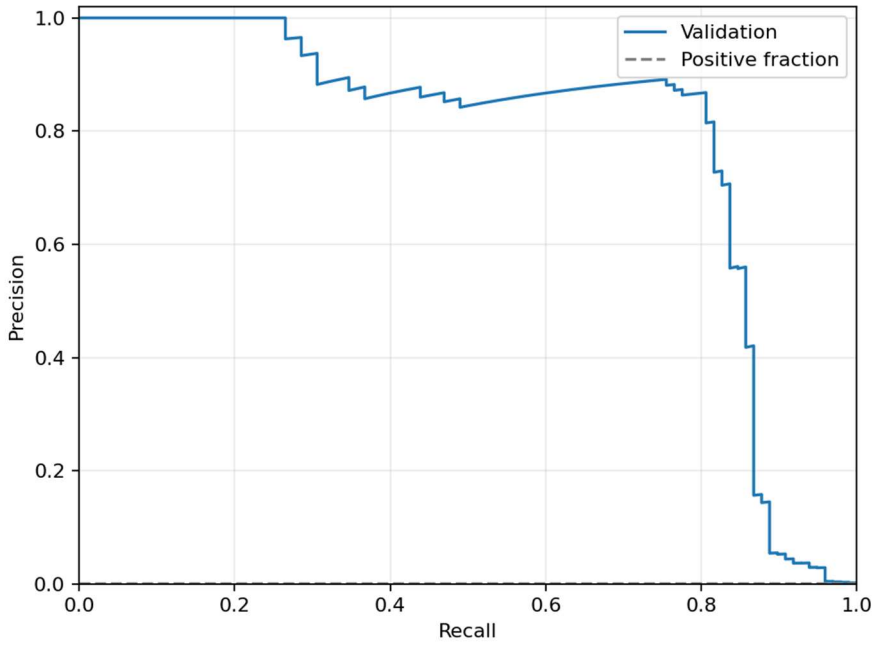
Еквівалентне трактування через ранжування, де P і N₀ - кількості позитивних і негативних прикладів:

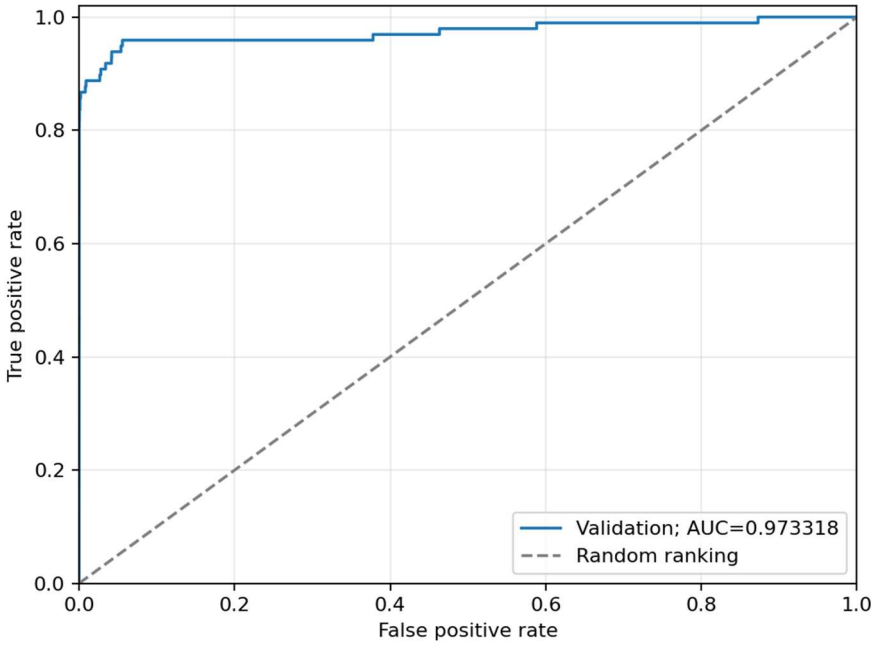
$$AUC = \frac{1}{PN_0} \sum_{i:y_i=1} \sum_{j:y_j=0} \left(\mathbf{1}[s_i > s_j] + \frac{1}{2} \mathbf{1}[s_i = s_j] \right).$$

Висока ROC-AUC характеризує ранжування за всіма порогами; вона не враховує конкретний поріг або ціну FN.

При великій кількості легітимних транзакцій навіть малий FPR може означати багато FP і низьку precision. Сіра лінія на PR — частка позитивних π; діагональ ROC відповідає випадковому ранжуванню.

За збереженими оцінками validation отримано ROC-AUC = 0.97331771637307996.





6 Точний пошук порога

Множина кандидатів T = {усі унікальні s на validation} ∪ {+∞}. Для скінченних оцінок +∞ означає всіх негативних, мінімальна оцінка — всіх позитивних. Поріг:

$$t^* = max\{t \in T : C(t) = min_{u \in T} C(u)\}.$$

Сортуємо оцінки за спаданням: $s_{(1)} \geq \ldots \geq s_{(n)}$. Позначимо $P = \Sigma_i y_i$, $N_0 = n - P$, $S_k = \Sigma_{i=1}^{k} y_{(i)}$. Лише в кінці групи однакових оцінок для $t = s_{(k)}$:

$$TP_k = S_k, FP_k = k - S_k, FN_k = P - S_k, TN_k = N_0 - (k - S_k),$$

$$C_k = 10(P - S_k) + (k - S_k).$$

Для +∞: TP = FP = 0, FN = P, TN = N_0, C = 10P. Включення всієї групи рівних оцінок відповідає правилу ≥: неможливо довільно дати різні класи об'єктам з однаковим score. Сортування потребує O(n log n), кумулятивні суми й обхід кінців груп — O(n), додаткова пам'ять — O(n). Повного проходу по validation для кожного кандидата немає. min за ключем (C, -threshold) реалізує вибір найбільшого порога серед мінімумів.

У CSV зберігаються всі кандидати, в README — лише 0.5, t* та +∞. Вхідні оцінки не округлюються. Перетворення float32 → float64 в аналізі точно зберігає їхні значення.

Отриманий мінімум вартості та порівняння з двома базовими правилами:

Точний t* = 0.031194042414426804.

rule	threshold	TP	FP	FN	TN	C
standard	0.5	71	9	27	56854	279
optimal	0.03119404241	82	34	16	56829	194
all_negative	inf	0	0	98	56863	980

На validation вартість змінилася з 279 до 194: FP +25, FN -11. Повна таблиця кандидатів збережена у threshold_candidates.csv.

7 Незалежні перевірки та тестова оцінка

Перевірка пошуку порога

checks.py самостійно формує кандидати через set і для кожного порога проходить усі пари (y, s) звичайним циклом. Він не використовує кумулятивні суми чи лічильники швидкого алгоритму. Порівнюються всі TP/FP/FN/TN/C і вибрані пороги на наборах A/B/C; збій зупиняє аналіз до оцінювання test.

A: пройдено, t* = 0.4. Усі TP/FP/FN/TN/C точно збіглися з незалежним перебором.

threshold	TP	FP	FN	TN	C
inf	0	0	2	4	20
0.9	0	1	2	3	21
0.7	1	2	1	2	12
0.4	2	2	0	2	2
0.2	2	3	0	1	3
0.1	2	4	0	0	4

B: пройдено, t* = inf. Усі TP/FP/FN/TN/C точно збіглися з незалежним перебором.

threshold	TP	FP	FN	TN	C
inf	0	0	0	3	0
0.9	0	1	0	2	1
0.6	0	2	0	1	2
0.3	0	3	0	0	3

C: пройдено, t* = 0.8. Усі TP/FP/FN/TN/C точно збіглися з незалежним перебором.

threshold	TP	FP	FN	TN	C
inf	0	0	2	11	20
0.8	1	0	1	11	10
0.5	2	10	0	1	10
0.2	2	11	0	0	11

A перевіряє спільне включення рівних оцінок 0.7; B — вибір +∞ без позитивних міток; C — однакову вартість 10 при 0.8 і 0.5, з вибором більшого порога 0.8.

Оцінювання на test

Після перевірок t* записується у threshold.json, потім застосовується до test. За формулами розділу 4 отримано такі метрики та матрицю плутанини:

threshold	TP	FP	FN	TN	C	precision	recall
0.03119404241	80	43	19	56820	233	0.6504065041	0.8080808081

Матриця: рядки — істинні класи 0, 1; стовпці — прогнози 0, 1.

true_class	predicted_0	predicted_1
0	56820	43
1	19	80

Описове порівняння з наперед заданим 0.5 (без зміни t*):

rule	threshold	TP	FP	FN	TN	C
standard	0.5	71	8	28	56855	288
fixed t*	0.03119404241	80	43	19	56820	233

Аналіз помилок FP та FN

Для FP і FN незалежно вибираються до п'яти об'єктів за зростанням початкового номера CSV. Для кожного наводяться y, s, сирий Amount і відстань:

$$d_i = |s_i - t^*|.$$

Для порога +∞ відстань нескінченна. Таблиця прикладів не дає підстав встановлювати причини помилок моделі.

row_id	error	label	score	Amount	distance
460	FP	0	0.1961589158	2	0.1649648733
2961	FP	0	0.08096113801	2.49	0.0497670956
10783	FP	0	0.9663270712	89.99	0.9351330288
13405	FP	0	0.9320316315	89.99	0.9008375891
13617	FP	0	0.9227622151	89.99	0.8915681727
10497	FN	1	0.011464376	3.79	0.01972966641
52521	FN	1	0.006373126525	105.99	0.02482091589
55401	FN	1	0.00564156333	1	0.02555247908
91671	FN	1	0.0005004229024	290.18	0.03069361951
93486	FN	1	0.01835153811	0	0.01284250431

Для наведених FP: Amount від 2 до 89.99; |s − t*| від 0.0497670956 до 0.9351330288.

Для наведених FN: Amount від 0 до 290.18; |s − t*| від 0.01284250431 до 0.03069361951.

Це перші помилки за порядком CSV, а не випадкова репрезентативна вибірка. За ними не можна встановити причини помилок чи залежність помилок від Amount. Відстань до порога описує близькість рішення до зміни класу, але не доводить каліброваність оцінок.

Висновок щодо вибору порога

На validation зміна порога з 0.5 на t* змінила FP на +25, FN на -11, C на -85. На test відповідні зміни: FP +35, FN -9, C -55.

Ваги моделі та її ранжування не змінилися; змінилося правило переведення оцінки у клас. За зниження порога більше транзакцій визнаються підозрілими: FN не зростає, але FP не зменшується. Мінімум на validation не гарантує такого

самого виграшу на test через інший набір транзакцій, малу кількість шахрайств та вибір порога саме за validation.

8 Запуск на Windows та macOS

Перший запуск потребує інтернету тільки для відсутніх бібліотек/Python. Файл uv.lock фіксує транзитивні залежності. Підготовка середовища:

uv sync --locked

Дві основні команди, однакові на Windows і macOS:

1. Відтворення результатів зі збережених прогнозів без повторного навчання:

uv run --locked python analyze.py --saved results --out results

Команда виконує перевірки A/B/C, пошук оптимального порога на validation, побудову PR- і ROC-кривих, обчислення ROC-AUC, оцінювання test та аналіз помилок FP/FN. Початковий файл creditcard.csv не потрібен. Очікувані результати: t* = 0.031194042414426804, вартість на validation — 194, на test — 233.

2. Повторне навчання моделі з локального CSV:

uv run --locked python train.py --data ../creditcard.csv --out results_retrained

Шлях ../creditcard.csv означає, що датасет розташований на один рівень вище папки lab2. За потреби його слід замінити власним шляхом у лапках. Команда виконує підготовку даних і навчання протягом 12 епох, після чого зберігає ваги та прогнози. Папка results_retrained повинна бути відсутньою або порожньою.

3. Аналіз результатів повторного навчання:

uv run --locked python analyze.py --saved results_retrained --out results_retrained

Нові таблиці, метрики та графіки зберігаються в results_retrained. Початкові результати в results залишаються незмінними. Після повторного навчання оцінки моделі можуть дещо відрізнятися між платформами; поділ даних і статистики стандартизації мають відтворюватися.

4. Окремий запуск перевірок алгоритму на наборах A/B/C:

uv run --locked python checks.py

9 ВИСНОВКИ

У ході лабораторної роботи навчено нейронну мережу для виявлення шахрайських транзакцій та досліджено, як поріг класифікації впливає на вартість помилок. **ROC-AUC на validation становить 0,9733**, що свідчить про

хороше ранжування транзакцій. Проте це значення не гарантує низької вартості помилок або високої precision при конкретному порозі.

За допомогою сортування та кумулятивних сум знайдено оптимальний поріг $(t^* \approx 0,031194)$. Порівняно зі стандартним порогом 0,5 отримано:

Показник на validation	**Поріг 0,5**	**Поріг (t^*)**
Знайдені шахрайства - TP	71	82
Хибні тривоги - FP	9	34
Пропущені шахрайства - FN	27	16
Precision	88,75%	70,69%
Recall	72,45%	83,67%
Вартість (10FN+FP)	279	**194**

Таким чином, на validation вартість зменшилася на **85 одиниць, або 30,47%**. Модель знайшла на 11 шахрайств більше ціною 25 додаткових хибних тривог. За заданих штрафів це вигідно, оскільки пропущене шахрайство коштує в десять разів дорожче за хибну тривогу.

PR-крива показує, що збільшення повноти виявлення шахрайств супроводжується зниженням precision. ROC-крива проходить значно вище діагоналі випадкового ранжування. Водночас при вибраному порозі FPR становить лише **0,0598%**, але це вже **34 хибні тривоги**. Через сильний дисбаланс класів навіть така мала частка помилок серед легітимних транзакцій помітно знижує precision.

Правильність алгоритму пошуку порога підтверджено незалежними перевірками **A/B/C**: усі лічильники та вартості точно збіглися з прямим перебором. Коректно оброблено однакові оцінки, відсутність позитивних міток і вибір більшого порога за однакової мінімальної вартості.

На незалежній тестовій вибірці зафіксований поріг дав **TP = 80, FP = 43, FN = 19, TN = 56 820**, precision — **65,04%**, recall — **80,81%**. Порівняно з порогом 0,5 вартість зменшилася з **288 до 233**, тобто на **19,10%**. Було пропущено на 9 шахрайств менше, але отримано на 35 хибних тривог більше.

Отже, **зміна порога дозволила зменшити вартість помилок без зміни ваг і повторного навчання моделі**. Покращення підтвердилося на test, хоча було меншим, ніж на validation. Це показує, що поріг, оптимальний на validation, не гарантує такого самого виграшу на нових даних.

9 Джерела та інструменти

1. «ЛР2 завдання» та lab2_starter.py.
2. Набір Credit Card Fraud Detection.
3. Перша лабораторна.
4. OpenAI Codex використано для підготовки коду, перевірок та звіту.